Thomas Nguyen
11888002
Professor Zhe Dang
Due 3/30/26

---

1. Let G be a DAG (a graph without loops) and u, v be two designated nodes (there are many other nodes in G). Design an efficient algorithm to count the number of paths from u to v.

```
def countPaths(graph, u, v)
    count = [0] + len(graph)
    count[v] = 1
    for node in topologicalOrder(graph)
        for neighbor in graph[node]
            count[neighbor] += count[node]
        return count[v]
```

```
def topologicalOrder(graph)
    visited = set()
    order = []
    def dfs(node)
        visited.odd(node)
        for neighbor in graph[node]
            if neighbor not in visited
            dfs(neighbor)
        order.append(node)
    for node in range(len(graph))
        if node not in visited
        dfs(node)
        return order(::-1)
```

2. Let G be a DAG (a graph without loops) and u, v be two designated nodes (there are many other nodes in G). In particular, each node in G is labeled with a color and multiple nodes can share the same color. A good path is one where the number of green nodes is bigger than the number of yellow nodes. Design an efficient algorithm to count the number of good paths from u to v.

```
def countGoodPaths(graph, colors, v, v)
    greenCount = [0] * len(graph)
    yellowCount = [0] * len(graph)
    for node in topologicalOrder(graph)
        for neighbor in graph[node]
        if colors[neighbor] = 'green'
        greenCount[neighbor] += greenCount[node] + 1
```

```
        yellowCount[neighbor] += yellowCount[node]
    elif colors[neighbor] = 'yellow'
        greenCount[neighbor] += greenCount[node]
        yellowCount[neighbor] += yellowCount[node] + 1
        goodPaths = 0
        for node in range(len(graph))
            if greenCount[node] > yellowCount[node]
                goodPaths += 1
            return goodPaths
```

```
def topologicalOrder(graph)
    visited = set()
    order = []

    def dfs(node)
        visited.add(node)
        for neighbor in graph[node]
            if neighbor not in visited
                dfs(neighbor)
            order.append[node]
    for node in range(len(graph))
        if node not in visited
            dfs[node]
        return order[::-1]
```

3. Let G be a graph (so it may have loops) and u, v be two designated nodes (there are many other nodes in G). In particular, each node in G is labeled with a color and multiple nodes can share the same color. Suppose that γis a regular expression on colors (e.g., (green + yellowyellow)∗yellowblue).An ugly path is one that the color sequence on the path satisfies the regularexpression γ. Design an efficient algorithm to count the number of uglypaths from u to v (when the count is infinite, return ∞). (Hint: you havetwo cases to consider, afer using a Cartesian product construction: a. thecount is infinite – where an SCC algorithm can be used to decide. b. thecount is finite, where prob 1 can be used.)
   For each SCC that contains a cycle:

- Check if the color sequence within it satisfies the regular expression $\gamma$
- If it does → **return ∞**
  If no SCC with a cycle satisfies $\gamma$, proceed to count finite ugly paths using dynamic programming. $dp(u, c)$ = number of paths from node $u \in V$ that end with color $c$ and satisfy $\gamma$ - $\text{next}(u, c)$ = set of nodes reachable from $u$ via an edge labeled with color $c$

$$dp(u,c) = \begin{cases} 1 & \text{if } u = v \text{ and } c \text{ satisfies } \gamma \\ 0 & \text{if } u = v \text{ and } c \text{ does not satisfy } \gamma \\ \displaystyle\sum_{u' \in \text{next}(u,c)} dp(u', c') & \text{if } u \neq v \text{ and } c \text{ satisfies } \gamma \\ 0 & \text{if } u \neq v \text{ and } c \text{ does not satisfy } \gamma \end{cases}$$

If any infinite paths exist, the count is $\infty$. Otherwise, use the DP above to compute the total count of finite ugly paths from $u$ to $v$.

3. Let G be a graph that may contain loops and hence, the number of pathsfrom a designated start node to a designated end node may be infinite. Un-fortunately, you usually can't say that one infinite number is larger than another. Here is the problem: sketch a way to compare the number of pathsin two graphs (that both may contain loops). (Hint: google perron, graph, path count).
   Let $A_1$ and $A_2$ denote the adjacency matrices of the two graphs respectively.
   **Perron's Theorem** guarantees the existence of a unique largest real eigenvalue for non-negative irreducible matrices, which can represent strongly connected graphs.
   If $\lambda_1 > \lambda_2$, it suggests that, on average, there are more paths in the first graph compared to the second graph.
   If $\lambda_1 < \lambda_2$, there are more paths in the second graph compared to the first graph.
   If $\lambda_1 = \lambda_2$, both graphs have a similar number of paths.
4. Let G be a control flow diagram of a C-progrm (which can be automaticallygenerated). For each node u in the diagram, one can obtain the total numberC(u) of paths from the root of the diagram to u. Then, from what you got from 4 above, you may sort all the C(u)'s for all u (even though some of C(u)'s are infinite) and then pick a u$*$ that has the maximal C(u). Write a mini-paper on how this will address the problem of testing a C-program. (if you want, you can actually publish a paper on this get a Master degree!)
   **Path Enumeration** For each node $u$ in the Control Flow Diagram (CFD), we compute $C(u)$ — the total number of paths from the root to $u$. While some paths may be infinite due to loops, the count for each individual path can still be obtained and reasoned about.
   **Maximal Path Node** $u^*$ By sorting all $C(u)$ values across every node $u$, we can identify a node:

$$u^* = \arg\max_u \ C(u)$$

   This node represents the **most reachable point** in the program the critical junction visited by the largest number of distinct execution paths from the root. It is, in a sense, the most "load-bearing" node in the control flow.
   **Testing Strategy** Knowing $u^*$, we can design **targeted test cases** that cover the diverse scenarios and branches leading to it. By focusing testing efforts on this critical node, we maximize **code coverage efficiency** exercising the greatest number of paths with the least redundancy.
5. (a job interview question for a top tech company) Design an efficient algorithm to compute the number of binary strings with length n that satisfy 1 the regular expression ((0 + 11 + 101)$*$(1101))$*$. (Hint: use Prob 3 above. I will talk about the solutions in class.)
   let DP(i) berive the # of binary strings of length *i* satisfy the given expression

$$DP[i] = DP[i-1] + DP[i-2] + DP[i-3] + DP[i-4]$$

   The best cases are $DP[0] = 1$ (which are empty strings) and $DP[0] = 0$ for $i < 0$